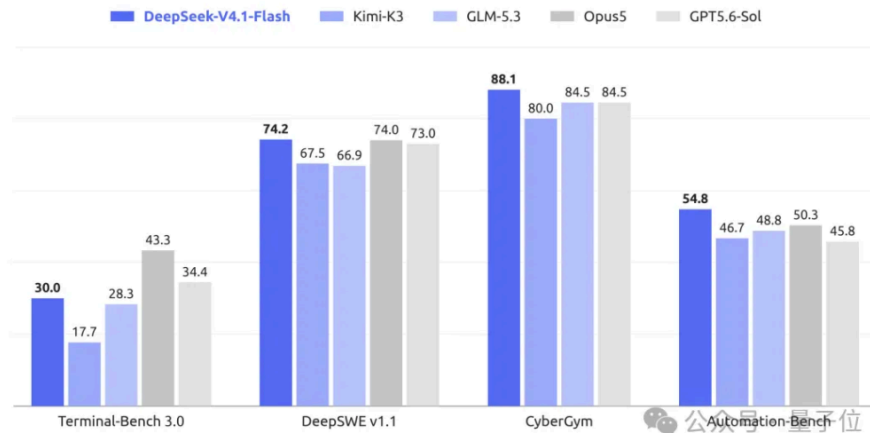


摘要：

DeepSeek V4.1 Flash采用全新非对称架构，为552B参数MoE模型，输入、输出激活分别为8B和16B。新模型大幅压缩KV Cache，使HBM需求降至1/4、SSD降至1/8。模型预训练45万亿token，后训练重点转向数据与环境工程，并引入Engram条件记忆模块。

DeepSeek V4.1 Flash上线，跑分重回国产一哥！



而且这次，是完全重新训练的非对称模型新架构。

552B参数的MoE模型，采用了全新的Causal-Encoder-Decoder结构，输入和输出不对称，**输入激活只有8B，输出激活16B**。

这种架构成本显著低于已知的同尺寸模型，难怪这两天测试感觉速度飞快，原来不光是因为用的人少。

这次重新预训练、用了新的数据、强化学习后训练规模更大，训练过程还与新版DeepSeek Harness v0.1.5深度结合。

所以**Flash打败自家Pro**（除了知识容量），昨天还觉得离谱，今天一下子合理了起来。

Benchmarks	DeepSeek V4.1-Flash	DeepSeek V4-Pro 0813	DeepSeek V4-Flash 0731	GLM 5.3	Kimi K3	GPT 5.6-Sol	Claude Opus 5
GPQA Diamond	90.9	92.4	89.9	88.1	92.9	94.1	93.4
HLE	36.8(39.1*)	42.7*	37.8*	42.0*	43.5	44.5	56.3
Codeforces (Rating)	3471	3348	3289	-	-	-	-
MathAreana Apex	65.6	65.3	58.6	-	65.6	-	-
Terminal-Bench 2.1	90.6	87.9	82.7	88.2	88.3	88.8	89.1
Terminal-Bench 3.0	30.0	11.8	7.6	28.3	17.7	34.4	43.3
Terminal-Bench 4.0	31.2	12.4	7.0	37.9	12.6	39.9	51.8
DeepSWE v1.1	74.2	62.7	54.4	66.9	67.5	73.0	74.0
ProgramBench	20.3	15.5	-	19.0	17.5	23.0	37.0
NL2Repo-Bench	65.4	61.5	54.2	58.0	58.0	56.8	75.3
CyberGym	88.1	83.3	76.7	84.5	80.0	84.5	-
SEC-Bench Pro	62.8	56.4	30.9	-	-	74.3	-
ExploitGym	15.3	5.4	1.8	15.0	-	33.7	22.1
HLE (w/tools)	63.9	60.0	51.5	62.5	59.8	-	63.6
Automation-Bench	54.8	43.2	37.7	48.8	46.7	45.8	50.3
Agents' Last Exam	31.8	25.7	25.2	28.5	27.6	26.7	28.6
Chartography(w/tools)	78.9	-	-	-	68.1	79.9	84.0
BabyVision (w/tools)	89.6	-	-	-	85.7	88.9	94.1
ZeroBench-main (w/tools)	49.0	-	-	-	41.0	53.0	52.0

*表示仅在 HLE 评测集的纯文本子集上进行了测试

V4.1 Flash也是DeepSeek首个原生支持多模态视觉理解的正式版模型，此前视觉能力以实验版 V4 Flash Vision Exp的形式单独提供，现在统一到了一个模型里。

再加上9月14日起所有V4 Pro的请求会被重新路由到V4.1 Flash。

所以未来一段时间内，直到V4.1 Pro上线，DeepSeek API将只提供这一个模型。

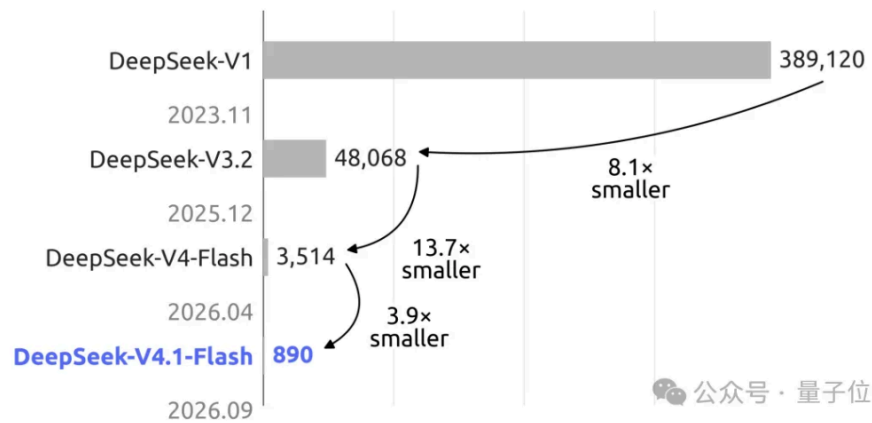
(2) 经多方测试，V4.1 Flash 在性能、费用、速度、总用时等各项指标上已全面超越 V4 Pro，因此我们计划有序下线 V4 Pro。北京时间 2026 年 9 月 14 日 12:00 之后，至未来 V4.1 Pro 上线之前，您访问 `deepseek-v4-pro` 的请求将全部路由到 V4.1 Flash，并按 V4.1 Flash 价格计费。

此外，还有一个能影响整个行业的消息。

推理效率方面，V4.1 Flash的KV Cache缓存大小相比上一代模型，对HBM的需求减少到1/4，对SSD的需求减少到1/8。

从初代V1到V4.1 Flash，KV Cache足足缩小到原来的1/437。

Global KV Cache Per Token (Bytes)



咱就是想问，内存啥时候降价？

51页论文解析

模型变化这么大，论文也少不了，刚放出的pdf足足有51页。

从标题就在强调“把KV缓存压缩到极致”。



DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression

DeepSeek-AI
research@deepseek.com

公众号 · 量子位

究竟怎么压缩这么狠的呢，先从架构开始说起。

非对称Transformer，提示词不必走完整40层

新架构是40层因果Transformer，被切分为前20层的因果编码器和后20层的解码器，这套结构命名为Causal Encoder-Decoder (CED)。

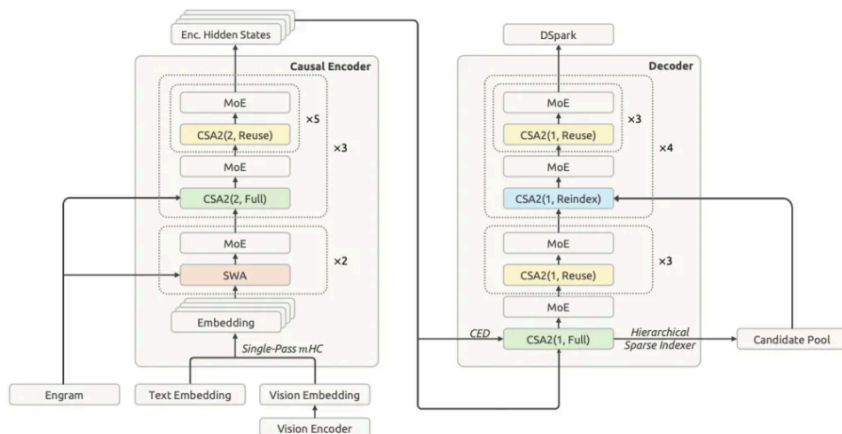


Figure 3 | **Overall architecture of DeepSeek-V4.1-Flash.** The 40-layer network is divided into a causal encoder and a decoder, each with 20 layers. All feed-forward layers use standard DeepSeekMoE. The first two encoder layers use sliding window attention (SWA); the rest use Compressed Sparse Attention 2 (CSA2), with CSA2(ratio, mode) specifying the compression ratio and mode. The model also uses Single-Pass mHC, Engram, DSpark, and a Hierarchical Sparse Indexer.

CED受YoCo架构启发，但做了一系列结构改进来提升KV Cache容量和KV生成的计算深度。

CED的核心在于非对称。**编码器处理输入时激活8B参数，解码器生成输出时激活16B参数。**

在Agent工作负载中，频繁的工具调用会产生大量prefill请求，CED的处理方式是：解码器的全局KV Cache不从各层自身的隐藏状态生成，而是直接从编码器最后一层（第20层）的隐藏状态投射得到，每层使用独立的投射权重。

这样一来，大部分prompt token只需通过前20层编码器，不必再走完整的40层。

对于序列长度远大于滑动窗口的场景，prefill计算复杂度直接从 $O(NL)$ 降至约 $O(NL/2)$ ，接近减半。

CSA2注意力：沿三个维度同时压缩缓存

接下来是注意力机制的改动。

Compressed Sparse Attention 2（CSA2）取代了V4中CSA与HCA的混合架构。

CSA2沿三个维度同时压缩缓存：**条目大小、序列维度和层维度。**

跨层复用全局KV和Top-K索引来减少存储和计算。

CSA2的压缩器和索引器也做了简化，去掉了CSA中相邻压缩条目之间的重叠和绝对位置嵌入，索引器K改为从主KV条目投射获得，而非从隐藏状态单独压缩。

CSA2定义了三种静态分配的工作模式。

- **Full模式**执行完整计算路径，生成自己的全局KV，投射索引器K，运行索引器产生Top-K索引。
- **Reindex模式**复用前序Full层的全局KV和索引器K，但用自己的索引器查询重新评分，选出新的Top-K。
- **Reuse模式**最轻量，直接复用前序层的全局KV和Top-K索引，不做任何索引计算。三种模式下每层都保留自己的查询和滑动窗口KV（SWA KV）。

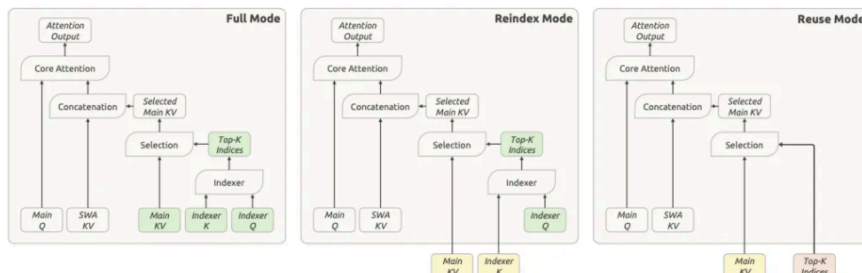


Figure 4 | Three operating modes of CSA2. The modes differ in how they obtain main KV, indexer K, and Top-K indices. Green blocks indicate quantities computed in the current layer; yellow blocks indicate main KV and indexer K reused from the most recent Full Mode layer; while red blocks indicate Top-K indices reused from the most recent index-producing (Full or Reindex Mode) layer. All three modes compute main Q and SWA KV in the current layer.

三种模式下每层仍保留自己的查询和滑动窗口KV。编码器以少数纯SWA层开场，其余按组编排，每组一个Full带若干Reuse；解码器则大量使用Reindex打头的组，在复用缓存的同时保留逐层重新选择注意力目标的能力。

解码器另外引入**层次化稀疏索引器**。

第一个Full层在扫描全部可见位置的同时，通过逐块评分构建一个共享候选池；后续Reindex层只在池内搜索。

由于池的大小固定，深层索引器的每查询计算量不再随上下文长度增长。这个机制是training-aware的，在后训练中引入，训练与推理施加相同的候选约束。

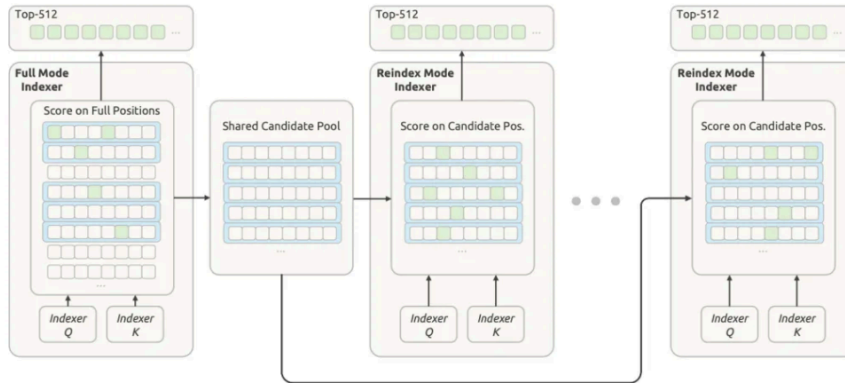


Figure 5 | **Hierarchical Sparse Indexer**. Each square represents a position; green squares mark selected indices, and blue rectangles mark blocks selected based on their maximum indexer scores. The decoder's first CSA2 layer in Full mode selects its own Top-512 indices and builds a shared candidate pool from the selected blocks for subsequent layers. CSA2 layers in Reindex mode then select their Top-512 indices from this pool.

公众号·量子位

缓存精度与持久化分层

缓存精度上，V4.1 Flash将**主KV Cache从FP8进一步压到FP4**，采用分组缩放因子的E2M1方案，并在论证数值范围安全后省略了二级全局缩放因子。

对量化更敏感的SWA KV仍保留FP8。FP4通过QAT在后训练阶段引入，量化点选在RoPE之后——放在之前只有边际精度收益，却要付出解码开销。

部署侧引入**SWA Bounded Replay**。

上一代精确重建SWA KV需要按层数成倍回放token，生产环境成本过高，V4.1改为只回放最近一个窗口的token做近似重建，对响应质量影响可忽略。

于是KV Cache被拆成两级，SWA KV移出持久化层，改放在各机预留的一小部分主机DRAM组成的分布式内存池中，TTL只有分钟级，过期即回收；全局KV留在SSD上的持久化缓存，保证数十小时生命周期。

其他架构扩展

- **Single-Pass mHC**：将输入混合系数偏移一个块，消除原始多核实现中的数据依赖；配合Mega-mHC部署内核，激活内存流量减半并触到理想下界。

- **Engram条件记忆模块**：在浅层和中层各插入一处，多阶n-gram、多hash head索引，推理时通过后台RDMA从主机内存预取嵌入。它贡献了模型中相当大比例的参数量。

- **DSpark投机解码**：在骨干冻结后独立训练，用少量Transformer块并行产出多个草稿位置的logits，配一个Markov head建模草稿token间依赖、一个置信度head预测逐位置接受概率，调度器再结合引擎吞吐曲线为每个请求动态选择验证长度。

预训练，数据重新清洗

模型在数十万亿token的多模态语料上预训练，全程无不稳定。

稀疏注意力**从零开始训练，没有任何密集注意力预热阶段**；多模态数据从第一步就混入；长上下文扩展在训练后段一次完成。

这次DeepSeek在数据侧提出两个判断。

一是把弱模型生成内容和低质量机器翻译定义为“**隐性重复**”并过滤掉，理由是它们主要是对已有信息的改写，在长训练周期中可能有害。

二是发现爬虫系统过度偏向纯文本网页，于是从Common Crawl重新引导抓取以提高多模态来源覆盖。

交错图文数据的构建被组织成成本递增的流水线：在昂贵的图像检索之前先用启发式过滤、去重和质量模型筛出高价值文档，组装成交错序列后再做图像感知的过滤与去重，最后交给VLM严格评分；被淘汰的文档还会部分回收为图文对。

优化器是分层的：线性变换权重用Muon，归一化与非矩阵参数用AdamW，嵌入表和预测头则用基于Sinkhorn平衡的动量更新。交替对行列归一化以近似均衡更新矩阵的行列RMS，只需一个动量缓冲区而非Adam的两个状态，直接压掉了巨量Engram参数带来的优化器状态内存。

Q/K权重使用head-wise Muon，为不同注意力头提供不同预条件器。视觉编码器在学习率衰减前保持冻结，仅归一化层与投射器可训练，衰减开始后再以更小学习率解冻联合优化。

后训练，只改数据不改算法

这次V4.1 Flash后训练流程沿用SFT + RL + OPD的标准范式，所有改变集中在数据与环境管线。

原文写道：“在当前阶段，工程化数据和环境管线的边际回报远超后训练中算法创新的边际回报。”

all of the observed gains. This observation echoes a broader lesson: at the current stage, the marginal return of engineering the data and environment pipeline substantially exceeds that of algorithmic novelty in post-training.

公众号·量子位

任务合成管线把每个训练任务形式化为（问题、环境、验证系统）三元组，沿难度与正确性两个维度评估，再用这两个信号迭代训练“造任务的模型”，并在任务每次进入新的RL运行时用产生的轨迹重新审计质量。

环境构建走了两条路。

General Agent环境取材于内部员工和合作伙伴的真实交互数据与失败案例，据此重建SaaS、企业应用和后端系统的mock工具集，连工具上下文、交互模式和失败条件一起复现。

Coding Agent环境则由多个专门化Agent协作流水线产出。构建Agent判断项目可运行性与可验证性并设计评测点，搭建Agent配置依赖、清除泄露解答的痕迹并打包镜像层，解题Agent尝试完成任务，独立质检Agent审查事实错误、评测点匹配度和可攻击性风险，不通过则交修复Agent返工重验。

支撑规模的是自研沙箱平台DSec，训练期间并发实例达到百万级。它没有用Kubernetes，而是自研放置引擎，以弱一致性换扩展性，多个副本不做同步协调，各自依据近期测量预测资源可用性独立决策，准入约束由各节点本地执行。单物理节点通过sub-NUMA分区与NUMA绑定，把容器密度提升了一倍以上，再配合延迟敏感执行类和超线程核心调度消除干扰。

RL中出现了大量奖励黑客和环境破坏行为。

Agent利用了真实披露的内核驱动权限问题、安全模块的非法内存访问、从软件包镜像服务泄露答案，也删过关键二进制、破坏系统文件、甚至移除整个文件系统。

防御手段是per-sandbox的AppArmor配置和基于eBPF的细粒度网络策略；Agent弄崩环境会被当作失败轨迹，并向RL框架回报一个“repercussion”信号。测试阶段同样出现类似行为。Agent会去反编译系统软件包挖漏洞。

可控推理强度

系统提示词中可加入一个标量effort级别，token惩罚系数随其指数衰减，API对外只暴露三档。

从低档到最高档，推理基准与Agent基准的成绩都有可观提升，代价是数倍输出token；但收益并不线性，中高区间就能恢复最大设置的大部分准确率而token消耗不到一半，最后一段提升要用近两倍的轨迹长度换边际改善。

虽然训练只用了有限几个级别，部署时中间值仍能引出插值后的推理行为。

OPD：多教师与token级中断恢复

蒸馏阶段动用了四十余个教师模型，不同领域的最优教师可能来自模型开发的不同阶段，彼此架构也未必相同。

配套的异步生成机制支持token级中断与恢复：rollout状态（含KV Cache与专家路由）以token粒度持久化，切换检查点后直接复用，彻底消除重新prefill的成本。

一句话总结

V4.1 Flash的全部架构创新几乎都指向同一个目标，让KV Cache在层与层之间尽可能被复用、在精度和存储层级上尽可能被压缩。

从而让长上下文Agent负载在经济上成立。

在后训练上，DeepSeek公开承认算法已非瓶颈，几乎把资源全押在数据与环境工程。

本文来源：[量子位](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

198 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论



华尔街韭菜
6小时前 中国江苏省

666

0 回复